Axionax Core Universe

Blockchain Core Only — High-Performance Protocol · DeAI · DePIN

> ***Scope:** This repository is the **blockchain core** of Axionax Protocol only. Frontend, website, marketplace, and SDK live in separate repositories (e.g. \[axionax-web-universe](https://github.com/axionaxprotocol/axionax-web-universe)).*

![CI](https://github.com/axionaxprotocol/axionax-core-universe/actions) ![License](#license) ![Rust](https://www.rust-lang.org/) ![Python](https://www.python.org/) ![EVM](https://ethereum.org)

PoPC Consensus · 45,000+ TPS · <0.5s Finality · DeAI at the Edge

Website · Documentation · Web Universe

Table of Contents

- About
- Quick Start
- Network (Testnet)
- Configuration
- Monolith Hardware
- Repository Structure
- Development
- Documentation
- Contributing

About

Axionax Protocol is a **DePIN (Decentralized Physical Infrastructure Network)** building a "Civilization OS" — turning Edge devices (Raspberry Pi, PC, Mac) into AI compute nodes on a high-performance blockchain.

Feature	Description
PoPC	Proof of Probabilistic Checking — statistical verification instead of full re-execution
DeAI	Python-based decentralized AI workloads via PyO3 bridge
Smart Contracts	WASM + EVM compatible
Geo-Hierarchy	5-tier network topology scaling toward 11M+ nodes

This repo contains: Node (Rust), RPC, consensus, staking, governance, DeAI worker (Python), configs, ops/deploy, and dev tools. It does **not** include the public website, dApp frontend, or marketplace UI — those are hosted and developed elsewhere.

Quick Start — Join the Network

> ***End users** → \axionax.org*
>
> ***Node Operators** — follow the steps below to run your own node.*

1. Clone & Update

```
git clone https://github.com/axionaxprotocol/axionax-core-universe.git
cd axionax-core-universe
python3 scripts/update-node.py
```

The script will:

- Create `.venv` automatically (handles Ubuntu 24.04 PEP 668)
- Install dependencies
- Run system suitability check (Python, deps, RPC)

2. Choose Node Type & Run

```
python3 scripts/join-axionax.py
```

Option	Type	Config
1	Worker (PC/Server)	`core/deai/worker\\\_config.toml`
2	Monolith Scout (single Hailo)	`configs/monolith\\\_scout\\\_single.toml`
3	HYDRA (Sentinel + Worker)	`configs/monolith\\\_worker.toml`

Or run directly:

```
# Worker
python3 core/deai/worker\\\_node.py

# Monolith Scout
python3 core/deai/worker\\\_node.py --config configs/monolith\\\_scout\\\_single.toml

# HYDRA (Sentinel + Worker dual-core)
python3 hydra\\\_manager.py
```

3. Update Node

Run on any machine that runs a node:

```
cd ~/axionax-core-universe
git pull
python3 scripts/update-node.py
```

For AI nodes (torch, numpy):

```
python3 scripts/update-node.py --full-deps
```

Current Network (Testnet)

Validator	IP	RPC	Region
#1	217.76.61.116	`http://217.76.61.116:8545`	EU
#2	46.250.244.4	`http://46.250.244.4:8545`	AU

- **Chain ID:** `86137`
- **Phase:** Pre-Testnet (Phase 2)
- Configs already point to these bootnodes

Verify RPC:

```
curl -s -X POST -H "Content-Type: application/json" \\\\
  -d '{"jsonrpc":"2.0","method":"eth\\\_blockNumber","params":\\\[],"id":1}' \\\\
  http://217.76.61.116:8545
```

Configuration

Config Files

File	Use Case
`core/deai/worker\\\_config.toml`	General Worker (PC/Server)
`configs/monolith\\\_scout\\\_single.toml`	Monolith Scout (single Hailo)
`configs/monolith\\\_sentinel.toml`	HYDRA — Sentinel (Hailo #0)
`configs/monolith\\\_worker.toml`	HYDRA — Worker (Hailo #1)

Environment Variables

```
cp core/deai/.env.example core/deai/.env
# Edit as needed
```

Variable	Description
`AXIONAX\\\_RPC\\\_URL`	RPC URL (overrides bootnodes)
`AXIONAX\\\_BOOTNODES`	Comma-separated RPC URLs
`AXIONAX\\\_CHAIN\\\_ID`	Chain ID
`AXIONAX\\\_WALLET\\\_PATH`	Path to wallet file
`WORKER\\\_KEY\\\_PASSWORD`	Wallet password (avoids prompt)
`WORKER\\\_PRIVATE\\\_KEY`	Private key (instead of file)

Security

- **Never commit** `.env` , `worker\\\_key.json` , or private keys (in `.gitignore`)
- **Backup wallet** after first run: `worker\\\_key.json` + password
- **Firewall:** only open necessary ports (Workers don't need 8545)
- **Production:** use `WORKER\\\_PRIVATE\\\_KEY` from env instead of file

Monolith MK-I Scout — Production

Hardware

Item	Notes
Raspberry Pi 5 (8GB)	Base unit
Raspberry Pi AI HAT+ 2 (Hailo-10H)	NPU for inference
Cooling	Heatsink/fan for Hailo thermal limits
SD card / SSD	Sufficient capacity + fast class
Power supply	5V 5A (USB-C PD)

Setup & Run

```
sudo apt update \\\&\\\& sudo apt upgrade -y
git clone https://github.com/axionaxprotocol/axionax-core-universe.git
cd axionax-core-universe
python3 scripts/update-node.py --full-deps

# Single Core (one Hailo)
python3 core/deai/worker\\\_node.py --config configs/monolith\\\_scout\\\_single.toml

# HYDRA (Sentinel + Worker — dual Hailo)
python3 hydra\\\_manager.py
```

Run as systemd Service

```
sudo cp scripts/axionax-hydra.service.example /etc/systemd/system/axionax-hydra.service
# Edit paths and user as needed
sudo systemctl daemon-reload
sudo systemctl enable --now axionax-hydra
```

Known Limitations

Item	Status
Worker registration / result submission	Mock — until smart contract deployed
Validator RPC	Live

Wallet / Keys	Live — creation and encryption work

Repository Overview

```
axionax-core-universe/
├── core/                       # Blockchain Protocol Core
│   ├── blockchain/             # Block and chain management
│   ├── consensus/              # PoPC consensus mechanism
│   ├── crypto/                 # Cryptographic primitives (Ed25519, Blake3)
│   ├── network/                # P2P networking + reputation system
│   ├── state/                  # RocksDB state management
│   ├── rpc/                    # JSON-RPC API + health endpoints
│   ├── staking/                # Native staking (stake, delegate, slash)
│   ├── governance/             # On-chain governance (proposals, voting)
│   ├── ppc/                    # Posted Price Controller
│   ├── da/                     # Data Availability (erasure coding)
│   ├── asr/                    # Auto-Selection Router (VRF worker selection)
│   ├── vrf/                    # Verifiable Random Function
│   └── deai/                   # DeAI (Python integration)
│
├── configs/                    # Monolith / Scout TOML configs
├── scripts/                    # Helper scripts (join, update, health-check)
├── ops/deploy/                 # Deployment \\\& Operations (Docker, monitoring)
└── tools/                      # Development utilities (faucet, devtools)
```

Key Features

- **High Performance:** 45,000+ TPS, <0.5s finality
- **PoPC Consensus:** Proof of Probabilistic Checking
- **Smart Contracts:** WASM + EVM compatible
- **DeAI Integration:** Python-based decentralized AI workloads
- **Native Staking:** Stake, delegate, slash, rewards
- **On-chain Governance:** Proposals, vote, execute
- **PPC / DA / ASR / VRF:** Dynamic pricing, data availability, worker selection

---

Development

Prerequisites

- Rust 1.70+
- Python 3.10+
- Docker & Docker Compose

Build & Test (Rust)

```
cd core
cargo build --release
cargo test --workspace
cargo clippy --workspace
```

```
cargo fmt --all
cargo bench
```

Run Local Node

```
cd core \\\&\\\& cargo run --bin axionax-node
```

Deploy with Docker

```
# Dev stack (local)
docker compose -f docker-compose.dev.yml up -d

# VPS
docker compose -f ops/deploy/docker-compose.vps.yml up -d
```

Python DeAI Tests

```
cd core/deai
python3 -m pytest . -v --tb=short --ignore=tests
```

Scripts Reference

Script	Description
`scripts/join-axionax.py`	System check + node type selection + run
`scripts/update-node.py`	Update node (git pull + deps + check)
`scripts/update-node.py --full-deps`	Update + AI/ML deps (torch, numpy)
`scripts/health-check.py`	Check RPC + config + wallet
`scripts/join-network.py`	Check config + RPC only
`scripts/verify-production-ready.py`	Full production readiness check
`scripts/make-node-package.py`	Create ZIP package for distribution

Troubleshooting

Issue	Solution
`pip` missing / PEP 668	`update-node.py` creates `.venv` automatically
Config file not found	Run from repo root or use `--config` with full path
No bootnodes	Set `\\\[network] bootnodes` in TOML or `AXIONAX\\\_RPC\\\_URL` in `.env`
Connection refused	Check RPC URL + firewall; verify chain is running

Wallet password	First run prompts; use strong password and store safely
`python` not found	Use `python3` (Ubuntu 24.04+)

Pre-launch Checklist

Area	Added	Location
Security	Audit scope + tooling (cargo audit, bandit)	docs/SECURITY_AUDIT_SCOPE.md, scripts/security/
Performance	TPS & finality load test (45k TPS, <0.5s)	scripts/load_test/
Infra	RPC multi-region, Explorer, Faucet, Monitoring	ops/deploy/environments/testnet/public/README_INFRA.md
Worker	LIVE contract integration, mock fallback	core/deai/CONTRACT_INTEGRATION.md

---

Documentation

Document	Description
Axionax Bible	เอกสารรวมหลัก — จุดเดียวสำหรับ Vision, Protocol, Run, Deploy, Launch
Genesis Public Testnet Plan	แผน launch testnet + จัดสรร VPS 3 ตัว
Add Network & Token (MetaMask)	เพิ่ม Axionax Testnet และ AXX ใน MetaMask / รับจาก Faucet
Connectivity Overview	การเชื่อมต่อ Local node, Validator, Frontend
GitHub Readiness	สถานะความพร้อม repo สำหรับ launch
Master Summary	Vision, architecture, hardware, tokenomics, roadmap
Self-Sufficiency	โปรโตคอลทำงานได้ด้วยตัวเอง ไม่พึ่ง external API ตอนรัน
Cyber Defense (DeAI)	ป้องกันภัยคุกคามทางไซเบอร์ผ่าน DeAI (7 Sentinels)
Architecture Overview	System architecture
API Reference	RPC API
Deployment Guide	Deployment instructions
Network Nodes	All node types
Core docs index	All docs in `core/docs/`

Contributing

1. **Fork** this repository

2. **Create** a feature branch (`git checkout -b feature/amazing`)
3. **Test** (`cargo test --workspace \\\&\\\& cargo clippy`)
4. **Push** and open a Pull Request

See CONTRIBUTING.md for details.

License

Component	License
core/	AGPLv3
ops/	MIT
tools/	MIT

Related Projects (separate repositories)

- **axionax Web Universe** — Frontend, SDK, Docs & Marketplace (website hosting and UI are done there; this repo is core only)

Support

Website · Docs · Issues

Built by the Axionax Protocol Team

Part of the Axionax Universe